Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Decorator kalıbı: amaç ve kalıtımla çözümün zayıflığı (ekrandan, 00:10:00)
  - Amaç: nesnelere çalışma anında, dinamik olarak ek sorumluluklar kazandırmak. Kalıtımla yapılan özelleştirme kodlama zamanında olur ve alt sınıfın bütün nesnelerini aynı biçimde etkiler; oysa aynı türden nesnelerin farklı sorumluluklar yürütebilmesi isteniyor.
  - GoF örneği: GUI çerçevesinde metin alanlarına yatay/dikey kaydırma çubuğu (scroll) ve çerçeve (border) yetenekleri. Kalıtımla çözüm: TextArea'nın alt sınıfları ScrollTextArea, BorderTextArea ve ikisini birleştiren SrcrollBorderTextArea.
  - Slayttaki zayıf yönler: Scroll ve Border yetenekleri iki ayrı yerde tekrar gerçeklenmek zorundadır; Panel sınıfına da aynı yetenekleri kazandırmak için ek üç sınıf daha gerekir.
  - Çoklu kalıtım her dilde yoktur, olsa da yalnız çeşitlilik sağlar, dinamiklik sağlamaz.
- Diamond (elmas) kalıtım problemi (00:10:06)
  - Sınıflar nokta, kalıtım okları kenar olarak çizilince elmas şekli çıkar. Üst sınıftaki draw() iki ara sınıfta override edilir; en alttaki sınıf override etmezse hangi ebeveynin draw() metodunu alacağı belirsizdir.
  - Olası kurallar: kalıtım listesinde ilk ya da son yazılanınki geçer ya da (hocanın C++ için anlattığı gibi) böyle metotların en alt sınıfta mutlaka override edilmesi zorunlu tutulur. Ek karmaşıklık getirdiği için güncel diller çoklu kalıtımı sınırlı destekler, arayüzler üzerinden gider.
- Decorator önerilen çözüm (ekrandan, 00:20:00)
  - Sınıfları değil, nesneleri birbiriyle ilişkilendirmek. Ortak soyut VisualComponent (Draw()) ve alt sınıfları TextView, (hocanın eklediği) Panel ve soyut Decorator.
  - Decorator, `component` adlı bir VisualComponent üyesi tutar; Draw() metodu `component->Draw()` yapar. Alt sınıfları ScrollDecorator (Draw(), ScrollTo(), scrollPosition) ve BorderDecorator (Draw(), DrawBorder(), borderWidth). BorderDecorator.Draw(): `Decorator::Draw(); DrawBorder();`.
  - Nesne düzeyinde zincir: aBorderDecorator → aScrollDecorator → aTextView. İstemci en dıştaki dekoratörle konuşur.
- Decorator kalıp yapısı (ekrandan, 00:22:00)
  - Component (Operation()), ConcreteComponent, soyut Decorator (component üyesi, Operation(): `component->Operation()`), ConcreteDecoratorA (addedState), ConcreteDecoratorB (Operation(): `Decorator::Operation(); AddedBehavior();`).
  - Ek davranış asıl işlemden önce, sonra ya da hem önce hem sonra yürütülebilir; bir koşula bağlanıp asıl işlem hiç yürütülmeyebilir.
  - Uygulanabileceği anlar: tek tek nesnelere dinamik ve şeffaf (aynı türden diğer nesneleri etkilemeden) sorumluluk eklemek; geri alınabilecek ya da askıya alınabilecek sorumluluklar; kalıtımdan uzak durulmak istenen durumlar.
  - Java örneği: stream tabanlı giriş-çıkış sınıfları (bir FileInputStream'i bir ObjectInputStream'e vererek dekore etmek).
- Decorator örnek gerçeklemesi: satış makbuzu (ekrandan, 00:30:00, 00:43:40, 00:44:30, 00:50:00)
  - Çözüm 1 (kullanım ilişkisi): TaskController → SalesOrder (process(), prtTicket()) → SalesTicket (prtTicket()) → Header (prtHeader()), Footer (prtFooter()). SalesTicket'ta if sınamalarıyla üstbilgi ve altbilgi basılıp basılmayacağına karar verilir.

  - Farklı üstbilgi/altbilgi türleri istenince soyut Header ve Footer ile alt sınıflar düşünülür; ama birden fazla üstbilginin birlikte istenmesi her kombinasyon için ayrı sınıf gerektirir.
  - Decorator çözümü (paket dp.decorator.example): soyut Component (prtTicket()), SalesTicket, soyut TicketDecorator (myComponent üyesi), Header1, Header2, Footer1, Footer2. İstemci için bir Factory sınıfı dekorasyon zincirini kurar.
  - Header'lar önce kendi adını basar sonra `super.prtTicket()` çağırır; Footer'lar önce `super.prtTicket()` çağırır sonra kendi adını basar.
- Java Reflection ile sınıf adını yazdırmak (00:40:52)
  - Her nesnenin Object'ten gelen getClass() metodu, sınıfını simgeleyen bir Class nesnesi döndürür. Bu nesneyle sınıfın adına (getName()), üye alanlarına ve metotlarına erişilebilir. Örnekte her prtTicket çağrısı `getClass().getName()` ile tam nitelikli sınıf adını (ör. `dp.decorator.example.SalesTicket`) yazar.
- Facade kalıbı (00:52:33)
  - Amaç: bir alt sistemdeki farklı arayüzleri tek ve daha üst düzey bir arayüzde toplamak; istemci karmaşık sistemin ayrıntılarını bilmeden tek bir erişim noktasından işini yapsın.
  - Benzetme: karmaşık bir bürokratik sistemde kapıdaki danışman. İsim, binanın dış cephe kaplamasından gelir.
  - Uygulanabileceği anlar: karmaşık bir sisteme basit bir arayüz sağlamak; sistemi katmanlara ayırmak; bağlaşımı azaltmak (istemciler yalnız Facade'a bağlı olur). Hoca basit olduğu için örnek gerçekleme yapmadı.
- Flyweight kalıbı (01:12:01, ekrandan 01:20:00)
  - İsim boks sıklet sınıflarından gelir (sinek sıklet). Amaç: paylaşım yoluyla çok sayıda basit nesnenin kullanımını desteklemek.
  - GoF örneği: metin editöründe her harfi grafik nesnesi olarak çizmek. Her konum için ayrı harf nesnesi çok bellek ve dosya boyutu harcar; bunun yerine harfler ortak bir havuzda tutulur, satır-sütun konumlarında havuzdaki nesneye referans verilir. Örnekte Row, Column ve Character sınıfları Glyph'ten türer.
  - Kalıp yapısı: FlyweightFactory (GetFlyweight(key): varsa var olanı döndür, yoksa oluştur, havuza ekle, döndür), soyut Flyweight (Operation(extrinsicState)), ConcreteFlyweight (intrinsicState), UnsharedConcreteFlyweight (allState), Client.
  - Intrinsic (içsel) durum nesnede saklanan, değişmeyen bilgidir (harfin bitmap resmi); extrinsic (dışsal) durum işlem için dışarıdan hesaplanıp verilen bilgidir (harfin satır ve sütunu).
  - Uygulanabileceği anlar: çok sayıda nesne kullanılıyorsa, saklama maliyeti yüksekse, durum içsel ve dışsal olarak ayrılabiliyorsa, dışsal durum ayrılınca az sayıda paylaşılan nesne çok sayıda nesnenin yerini alabiliyorsa ve uygulama nesne kimliğine bağlı değilse.
- Flyweight örnek gerçeklemesi: kimya fabrikası (ekrandan, 01:30:00, 01:40:00, 01:50:00, 01:52:00, 02:00:00, 02:10:00)
  - Problem: kimyasal maddeler tek başına ya da karışımlar halinde gram cinsinden satılıyor; bir madde (ör. karbon) birden fazla karışımda farklı gramajlarla bulunabilir.
  - İlk çözüm (dp.flyweight.solution1a/b/c): KimyasalMadde (gram, isim, simge, atomAgirligi; setGram, getGram, getIsim, getAtomAgirligi, getMol: `return gram/atomAgirligi;`) ve Karışım (isim, `maddeler : LinkedList<KimyasalMadde>`, setIsim, getIsim, maddeEkle, tarifEt).
  - Örnek: 100 gr barutta 75 gr nitrat, 15 gr karbon, 10 gr kükürt. Kara barut sodyum nitrat (NaNO3), temiz barut potasyum nitrat (KNO3) ile yapılır.
  - Çözüm a: her karışım için ayrı madde nesneleri; sonuç doğru ama aynı madde bilgisi çok yerde tekrarlanır. Çözüm b: ortak tek karbon nesnesi; ikinci karışımda setGram(14) birinci karışımdaki gramajı da bozar (iki karışımda da 14 görünür).
  - Daha doğru modelleme (dp.flyweight.solution2): gram bilgisi Bileşen sınıfına taşınır (gram, madde; getGram, getMol, getMaddeAdi). Karışım `bilesenler : LinkedList<Bileşen>` tutar, `bileşenEkle(madde, gram)` ya da `bileşenEkle(Bileşen)` sunar. KimyasalMadde'de getMol(gram) olur.
  - Flyweight çözümü (dp.flyweight.solution3): MaddeFabrikasi (`maddeler : HashMap<String, KimyasalMadde> {static}`, `maddeBul(simge) : KimyasalMadde {static}`); Karışım.bileşenEkle simge ve gram alır. Anahtar simge, gram dışsal bilgidir.

- Proxy kalıbı (02:11:10)
  - Amaç: bir nesneye erişimi denetlemek için onun yerine geçecek başka bir nesneyi (vekil) araya koymak.
  - Örnekler: belgede büyük resimlerin hızlı kaydırmada yer tutucuyla gösterilip sayfada durunca yüklenmesi; web sayfalarında resimlerin yerine önce yükleme simgesi gelmesi. GoF yapısında ImageProxy, Graphic ile aynı arayüze uyar.
  - Kalıp yapısı: istemci gerçek nesneyle (RealSubject) değil, ortak arayüz (Subject) üzerinden vekille (Proxy) konuşur; vekil isteği gerçek nesneye iletir, gerektiğinde araya girip ek davranış (ör. yükleme) sergiler.
  - Uygulanabileceği anlar: bir nesneye erişimin basit bir işaretçiden fazlasını gerektirdiği durumlar; uzak nesneler (başka bir bilgisayardaki Java sanal makinesinde çalışan nesnelere Java RMI ile erişim).
- Proxy örnek gerçeklemesi: izlediğin kadar öde sunucusu (ekrandan, 02:20:00, 02:22:30, 02:26:40)
  - Problem: sunucu verilen adrese istenen programı gönderir; göndermeden önce ödeme denetimi yapılmalı, ama sunucunun kaynak kodu yok. Araya konan vekil işi çözer (protection proxy).
  - Tasarım (dp.proxy.example): «interface» StreamSource (`stream(programID, destination : String) : boolean`), onu gerçekleyen StreamServer ve StreamProxy (`original : StreamSource`, `checkPayment(programID, destination) : boolean`). StreamDestination (server, address; makePayment(double), requestProgram(programID)). MainApp istemcidir.
  - StreamProxy.stream(): checkPayment başarılıysa `original.stream(...)` döndürür, değilse false döndürür.
- Chain of Responsibility kalıbı (02:27:44, ekrandan 02:30:00)
  - Davranışsal (behavioral) kalıplara geçiş. Amaç: birbiriyle ilişkili bir nesneler zinciri ya da hiyerarşisindeki bir nesnenin işleyemediği mesajı, işleyebilecek başka bir nesneye iletmek.
  - Mesajın işlenememesinin nedeni nesnenin arayüzünde o mesajın olmaması değil, nesnenin mesajı işleyecek yetki ya da bilgiye sahip olmamasıdır.
  - Örnek: bağlama duyarlı yardım. Yardım isteği tıklanan bileşende yanıtlanamazsa onu içeren üst bileşene (OK düğmesi → yazdırma iletişim kutusu → uygulama) iletilir; iletim mesaj yanıtlanana ya da zincirin sonuna gelene dek sürer.
  - Kalıp yapısı: soyut Handler (HandleRequest(), successor üyesi; üye alanı olduğu için arayüz olamaz) ve somut alt sınıfları.
  - Uygulanabileceği anlar: bir mesajı yanıtlayabilecek birden fazla nesne varsa ve yanıtlayacak nesne önceden bilinemiyorsa.
- Chain of Responsibility örnek gerçeklemesi: reklam yönetimi (ekrandan, 02:40:00, 02:45:50, 02:50:00, 02:53:00)
  - Kampanya 1: spor sitesinin 18-39 yaş arası erkek ziyaretçilerine tıraş bıçağı, 18-39 yaş arası kadın ziyaretçilerine fitness salonu, diğerlerine spor mağazası indirimi reklamı.
  - Kampanya 2: haber sitesinin 18-29 yaş arası A gelir grubu erkek ziyaretçilerine otomobil dergisi reklamı, aksi halde Kampanya 1 aynen uygulanır (alt zincirin yeniden kullanımı).
  - Tasarım (dp.chainOfResponsibility.example): User (Cinsiyet, GelirDuzeyi, yaş), enum Cinsiyet (KADIN, ERKEK), enum GelirDuzeyi (A, B, C), Banner, soyut Reklam (banner, sonraki; soyut reklamUygunMu(User), reklamGoster(User); final reklamKotar(User)), alt sınıfları TrasBicak, FitnessSalonu, IndirimSporMagaza, OtoDergi.

Hocanın Özellikle Vurguladığı Kısımlar

- Sınavda şemaları UML standardına göre çizin
  - GoF gösterimi UML'ye uymaz, çünkü o zaman standart oluşmamıştı. Kalıtım oku birleşim çizgisinin ortasında değil, üst sınıfa dayanacak biçimde çizilmeli (birleştirme/sahiplik gösterimiyle karışmasın); üye metotlar public, üye alanlar private işaretlenmeli (00:14:33).
- Uzun if-else zinciri tasarım hatasının işaretidir
  - "Şu ise header 1, değilse şu ise header 2…" gibi kod yazılıyorsa daha iyi bir tasarım

yapılabilir; kalıtım ve çok biçimlilik kullanılmalıdır (00:35:00).
- Decorator'da asıl işlemi çağırmayı unutmayın
  - Somut dekoratörün operasyonunda üst sınıfın (Decorator) operasyonu uygun bir yerde mutlaka çağrılmalıdır; çağrılmazsa dekore edilen bileşenin işi yapılmaz.
- Decorator'ın zayıf yönleri
  - Bir sorumluluk tamamen iptal edilirse zincirdeki diğer bileşenlerin kodu çağrılmayabilir, yanlış davranış ve hesaplamalar çıkabilir.
  - Nesneye en içteki değil, en son dekore eden (en dıştaki) nesne üzerinden erişilir. Zincirin ortasındaki bir nesneye erişmek için zinciri bozmak yerine, gereken ara halkayla yeni bir zincir kurmak daha doğrudur.
- Decorator'da sıra
  - Header'lar her zaman makbuzdan önce, Footer'lar her zaman sonra basılır; bu yüzden header-footer ilişkilendirme sırası çıktıyı etkilemez. Ama aynı amaca hizmet eden dekoratörlerin (iki header ya da iki footer) kendi aralarındaki ilişkilendirme sırası çıktı sırasını değiştirir.
- Decorator'da dekore edilen nesne null olabilir
  - Örnek kodda TicketDecorator.prtTicket() önce `myComponent != null` denetimi yapar; hocaya göre bu, kalıpta söylenene bir ektir.
- Flyweight'in bütün koşulları sağlanmalıdır
  - Durum bilgisi içsel ve dışsal olarak ayrılamıyorsa kalıp uygulanamaz; nesne sayısı çok değilse "atılan taş ürkütülen kuşa değmez". İçsel durum ne kadar büyükse tasarruf o kadar çoktur.
- Flyweight'ler bir bakıma Singleton'dır
  - Her harften (ya da her kimyasal maddeden) tek bir nesne vardır, ama Singleton kalıbını ayrıca kurmak şart değildir (01:23:24). İçsel durumu oluşturan alanlar final yapılabilir.
- Nesne kimliği gerekiyorsa paylaşım uygun olmaz
  - Durum bilgileri aynı olsa da farklı nesneler olduğu bilgisinin korunması gerekiyorsa UnsharedConcreteFlyweight kullanılır; bu durumda tasarruf azalır.
- Neyin değiştiğini bul ve onu sarmala
  - Kimya örneğinde değişen şey gramdır; gram madde sınıfından çıkarılıp Bileşen sınıfına taşınır (hoca bunu ileride görülecek Extract Field/Extract Class refactoring'ine bağladı).
- Proxy, tek kademeli bir Decorator gibi düşünülebilir
  - Ama esas amacı yeni yetenek vermek değil, nesneye erişimi denetlemektir (02:16:51).
- Chain of Responsibility'de zinciri doğru yapılandırın
  - Hiyerarşinin kökünde mesaj mutlaka ele alınmalıdır; aksi halde mesaj yanıtsız kalır. Reklam örneğinde zinciri hoca kendisi kurduğu için zincirin sonuna her kullanıcıya uygun bir reklam (spor mağazası indirimi) koydu.
- Zinciri yöneten metodu final yapın
  - reklamKotar metodu final işaretlenir; alt sınıflar bu çalışma mantığını override edip bozamaz.

Kısa Tekrar Notları

- Decorator: nesnelere çalışma anında ek sorumluluk. Roller: Component, ConcreteComponent, Decorator (component üyesi, işlemi ona iletir), ConcreteDecorator (önce/sonra ek davranış).
- Kalıtımla çözüm: yetenek kodu tekrarı, kombinasyon başına sınıf, dinamiklik yok; çoklu kalıtımda diamond problemi.
- Makbuz örneği: SalesTicket (ConcreteComponent), TicketDecorator, Header1/2 (önce kendini basar), Footer1/2 (sonra kendini basar), Factory zinciri kurar.
- `getClass().getName()` nesnenin tam nitelikli sınıf adını verir (Java Reflection).
- Facade: karmaşık alt sisteme tek, basit erişim noktası; bağlaşımı azaltır. Benzetme: kapıdaki danışman.
- Flyweight: çok sayıda ince taneli nesneyi paylaşmak. FlyweightFactory havuzu yönetir; intrinsic durum nesnede, extrinsic durum dışarıdan verilir.
- Kimya örneği: gram dışsaldır → Bileşen sınıfı; MaddeFabrikasi simgeye göre tek KimyasalMadde nesnesi döndürür.

- Proxy: erişimi denetleyen vekil; Subject arayüzü, RealSubject, Proxy. Türler örnekte: protection proxy, uzak nesneler (RMI), yer tutucu resim.
- Pay-per-view örneği: StreamProxy, checkPayment başarılıysa original.stream(...) çağırır.
- Chain of Responsibility: işlenemeyen mesaj successor'a iletilir. Handler soyut sınıftır (successor üyesi).
- Reklam örneği: reklamUygunMu ise göster, değilse sonraki reklamın reklamKotar'ını çağır.

Detaylı Açıklamalar

- Decorator'ın GoF şeması ve hocanın eklemeleri (ekrandan, 00:20:00)
  - Hoca şemaya Panel sınıfını (+Draw()) ekledi, Decorator'dan VisualComponent'e giden "component" ilişkisinin şemada parça-bütün gibi görünse de aslında bire bir olduğunu söyledi: bir dekoratör nesnesi yalnız bir VisualComponent'i süsler.
  - ScrollDecorator önce `Decorator::Draw()` ile bileşenin çizilmesine izin verir, sonra kaydırma çubuklarını ekler; çubuğa tıklama gelince ScrollTo ile ilgili satıra kaydırır. Hoca, yatay ve dikey ayrımı yapılsaydı HorizontalScrollDecorator ve VerticalScrollDecorator diye iki sınıf gerekeceğini belirtti.
  - TextView ScrollDecorator ile dekore edilince ikisi birlikte tek bir VisualComponent gibi davranır; üstüne BorderDecorator konunca daha büyük üçüncü bir bileşen oluşur.
- Önce ve sonra yürütülen ek davranış örneği (00:23:00)
  - İnsan kaynakları sürecinde bazı işlerden önce iş sağlığı ve güvenliği izni alınmalı, bazılarından sonra iş tamamlama tutanağı teslim edilmeli, bazılarında ikisi de gerekir. Hasta bir doktorun hasta kabul etmemesi gibi koşullu durumlarda dekoratör önce bir if ile denetler, koşul sağlanmazsa asıl işlemi yürütmez. Bu yalnız o doktorun rolünü askıya alır, diğer doktorlar etkilenmez.
- Java stream sınıfları (00:26:00)
  - Temel input stream yalnız bayt okur. Metin girişi için yararlı metotlar içeren bir stream sınıfına, kurulurken bir input stream verilir; nesne okumak için ObjectInputStream'e bir FileInputStream verilir. Dosya sıkıştırılmışsa araya bir zip input stream konur. Hoca sınıf adlarının tam olarak böyle olmayabileceğini söyledi; mantık, çekirdek bir yeteneği yeni yeteneklerle istediğiniz biçimde sarmaktır.
- Makbuz örneğinin kodu (ekrandan, 00:43:40, 00:44:30, 00:50:00)

```java
package dp.decorator.example;

public abstract class TicketDecorator extends Component {
    private Component myComponent;
    public TicketDecorator (Component myComponent) {
        this.myComponent= myComponent;
    }
    public void prtTicket () {
        if (myComponent != null) myComponent.prtTicket();
    }
}

public class Header1 extends TicketDecorator {
    public Header1 (Component myComponent) {
        super(myComponent);
    }
    public void prtTicket () {
        // place printing header 1 code here
        System.out.println(getClass().getName());
        super.prtTicket();
    }
}
```

```
public class Factory {
    public Component getComponent () {
        Component myComponent;
        myComponent = new SalesTicket();
        myComponent = new Footer1(myComponent);
        myComponent = new Footer2(myComponent);
        myComponent = new Header2(myComponent);
        myComponent = new Header1(myComponent);
        return myComponent;
        /*
        return new Header1( new Header2( new Footer2(
                new Footer1( new SalesTicket() ) ) ) );
        */
    }
}
```

  - Bu düzenin çıktısı: `dp.decorator.example.Header1, Header2, SalesTicket, Footer1, Footer2` (ekrandan, 00:50:00). Header1 en dışta olduğu için ilk basılır; Footer1 en içteki footer olduğu için önce basılır. Hoca Header1 ve Header2'nin sırasını değiştirince çıktının Header2, Header1 olduğunu gösterdi (00:47:34).
  - Factory Method fikri: farklı header-footer kombinasyonları için bir fabrika sınıfı; bu kod istemcinin içine de konabilirdi.

- Flyweight'in bellek kazancı (01:16:04)
  - Dosyada önce havuz kaydedilir, metinde her harfin yerine havuzdaki nesnenin sırası tutulur. Hocanın sayılarıyla: bir harf nesnesi onlarca, yüzlerce kilobayt olabilirken ona işaret eden bir tamsayı 4 bayttır.

- Kimya örneğinde çözüm a ve b (ekrandan, 02:00:00)
  - Çözüm a'da kara ve temiz barut için ayrı karbon nesneleri oluşturulduğundan biri 15, diğeri 14 gram olabilir; sonuç doğrudur ama her karışımda madde bilgisi tekrarlanır.
  - Çözüm b'de tek karbon nesnesi (c) paylaşılır: `c.setGram(14); temizBarut.maddeEkle(c);` satırı kara baruttaki 15 gramı da 14 yapar. Ekranda iki karışımda da "14.0gr Karbon" çıktısı görülür; hoca bunu "yanlış sonuç" olarak gösterdi. Çözüm a ideal değil ama en azından doğru sonuç verir.

- Flyweight çözümünün ana programı (ekrandan, 02:10:00)

```
package dp.flyweight.solution3;
public class MainApp {
    public static void main(String[] args) {
        Karisim karaBarut = new Karisim();
        karaBarut.setIsim("Kara barut");
        karaBarut.maddeEkle("NaNO3", 75);
        karaBarut.maddeEkle("C", 15);
        karaBarut.maddeEkle("S", 10);
        Karisim temizBarut = new Karisim();
        temizBarut.setIsim("Temiz barut");
        temizBarut.maddeEkle("KNO3", 75);
        temizBarut.maddeEkle("C", 14);
        temizBarut.maddeEkle("S", 11);
        System.out.println( karaBarut.tarifEt() );
        System.out.println( temizBarut.tarifEt() );
    }
}
```

  - Slayttaki şemada metot adı `bileşenEkle(simge, gram)`, kodda `maddeEkle(simge, gram)` olarak geçiyor.
  - MaddeFabrikasi'nda havuz ve maddeBul static yapılmıştır; bu, Singleton benzeri tek havuzla en fazla tasarrufu sağlar. Maddeler farklı gruplar halinde tutulmak istenirse

fabrikanın birden fazla örneği de olabilir. Fabrikaya üretilen maddelerin dökümünü veren bir listeleme metodu da eklenebilir.
  - Ders sonrası not: Kodda (ekrandan, 02:00:00) Potasyum Nitrat (KNO3) için de sodyum nitratın değeri olan 84.9947 verilmişti. KNO3'ün mol kütlesi büyük olasılıkla yaklaşık 101.10 g/mol'dür. Hoca aradan sonra atom ağırlıklarını düzelttiğini söyledi (02:09:50).

- Proxy örneğinin kodu (ekrandan, 02:26:40)

```
public StreamProxy(StreamSource original) {
    this.original = original;
}
public boolean stream(String programID, String destination) {
    if( checkPayment(programID, destination) )
        return original.stream(programID, destination);
    else
        return false;
}
public boolean checkPayment(String programID, String destination) {
    // Ödemeyi destination parametresine göre kontrol et
    // Ödeme yapılmışsa true, aksi halde false döndür.
    return false;
}
```

  - StreamServer iskelet olarak doğrudan true döndürür. Hoca vekili StreamServer türünden değil StreamSource arayüzü türünden bir üye ile kurdu; böylece aynı arayüze uyan başka sunucular da desteklenir, her sunucu için ayrı vekil gerekmez.

- Smalltalk'ta doesNotUnderstand (02:29:21)
  - Smalltalk'ta bir nesnenin sınıfında tanımlı olmayan bir metodu çağrılırsa doesNotUnderstand metodu çalışır ve gövdesine yazılan kod yürütülür. C#, C++ gibi dillerde böyle bir mekanizma yoktur; Chain of Responsibility'de iletilen mesaj zaten alıcının arayüzünde vardır.

- Reklam örneğinin kodu (ekrandan, 02:45:50, 02:50:00)

```
public Reklam(Banner banner, Reklam sonraki) {
    this.banner = banner; this.sonraki = sonraki;
}
public abstract boolean reklamUygunMu ( User u );
public abstract void reklamGoster( User u );
public final void reklamKotar( User u ) {
    if( reklamUygunMu(u) ) {
        System.out.println( this.getClass().getName()
                + "->" + u );
        reklamGoster(u);
    }
    else if( sonraki != null ) {
        System.out.println( this.getClass().getName()
                + "->" + sonraki.getClass().getName());
        sonraki.reklamKotar(u);
    }
}

public void kampanya1( ) {
    Reklam[] reklamlar = new Reklam[3];
    reklamlar[2] = new IndirimSporMagaza(new Banner(), null);
    reklamlar[1] = new FitnessSalonu(new Banner(), reklamlar[2]);
    reklamlar[0] = new TrasBicak(new Banner(), reklamlar[1]);
    reklamlar[0].reklamKotar(users[0]);
    reklamlar[0].reklamKotar(users[1]);
    reklamlar[0].reklamKotar(users[2]);
```

```
}
```

  - Kullanıcılar: users[0] ERKEK, C, 23; users[1] ERKEK, A, 53; users[2] KADIN, B, 33; users[3] ERKEK, A, 28 (ekrandan, 02:53:00). Zincir sondan başa kurulur, çünkü her reklam bir sonrakini kurucusunda alır.
  - TrasBicak.reklamUygunMu erkek ve 18-39 yaş aralığını, FitnessSalonu kadın ve 18-39 yaş aralığını denetler; IndirimSporMagaza doğrudan true döndürür.
  - Kampanya 1 çıktısı (ekrandan, 02:53:00): users[0] TrasBicak'ta; users[1] TrasBicak → FitnessSalonu → IndirimSporMagaza'da; users[2] TrasBicak → FitnessSalonu'nda reklam görür.
  - Kampanya 2'de dört elemanlı dizinin başına OtoDergi konur, geri kalan zincir aynıdır. users[0] gelir grubu C olduğu için otomobil dergisini görmez, tıraş bıçağına geçer; users[1] yaşı nedeniyle dergiyi görmez, indirime kadar iner; users[3] (ERKEK, A, 28) otomobil dergisi reklamını görür.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.